

Seed Data

Run `npm run seed` (from `backend/`) after migrating. All seeded users share one password: `Password123!`

All timestamps in the seed script are computed relative to the moment it runs (`new Date()` at the top of the script), not hardcoded — re-running `npm run seed` at any point in the future reproduces the same five auction states, not five stale ones.

Users

Email	Role	Name
buyer1@rfq-demo.com	Buyer	Ananya Rao
buyer2@rfq-demo.com	Buyer	Rohan Mehta
supplier1@rfq-demo.com	Supplier	Vikram Singh — bids as Global Cargo Lines
supplier2@rfq-demo.com	Supplier	Meera Iyer — bids as Swift Freight Solutions
supplier3@rfq-demo.com	Supplier	Arjun Nair — bids as Oceanic Shipping Co.
supplier4@rfq-demo.com	Supplier	Kavita Desai — bids as TransWorld Logistics
supplier5@rfq-demo.com	Supplier	Rahul Kapoor — bids as Horizon Freight Services

(Carrier names are fictional; any resemblance to real freight forwarders is coincidental — they exist only to make demo bids readable.)

RFQs

Deliberately covers every status the system can produce, including the internal-only `Scheduled` state (real status, computed server-side — see `HLD.md` § 3 for how status is derived):

Reference	Status	What it demonstrates
RFQ-2026-001	Scheduled	Starts 2 days out — the pre-start bidding guard
RFQ-2026-002	Active	3 bids in, already extended twice via rank-change triggers, still open
RFQ-2026-003	Active	Zero bids — the empty state
RFQ-2026-004	Closed	2 bids, closed naturally before ever reaching its forced ceiling

RFQ-2026-005	Force Closed	4 bids, extended repeatedly (any-bid trigger) until it hit the forced close ceiling
--------------	--------------	---

Bid totals in every seed row are computed via the same `computeTotalAmount` helper the live bid-submission path uses (`src/lib/auction-rules.ts`) rather than hand-typed sums, so seed data can't silently drift from how the real app computes a bid total.